Exploiting HDMI and USB Ports for GPU Side-Channel Insights

Sayed Erfan Arefin Abdul Serwadda

Department of Computer Science, University of Dayton & Computer Science Department, Texas Tech University

CODASPY '25: Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy

Abstract

Modern desktops and workstations expose numerous USB and HDMI ports. We show that these ports inadvertently leak information about **GPU activity** through **power fluctuations** observable by a small external device. Our prototype:

- Plugs into **USB** or **HDMI** ports.
- Measures high-resolution current traces using a **Hall-effect** sensor.
- Uses **ML classifiers** to infer GPU workloads such as CNN architectures and video rendering.

Result: a new, stealthy power side-channel that works without software on the victim machine and without direct access to the GPU or PSU.

- GPUs drive **DNN inference**, graphics, and video rendering, often on sensitive data.
- Prior power side-channel work typically:
  - Taps **GPU/CPU rails** (oscilloscope + shunt),
  - Or uses privileged **power interfaces** (e.g., nvidia-smi, RAPL).
- USB/HDMI ports are viewed mainly as:
  - *Data-plane* attack surfaces (BadUSB, firmware attacks),
  - Not as *power side-channel* observation points.

Key Question: Can an unprivileged device on HDMI/USB observe enough power variation to infer *which GPU workloads* are running?

Threat Model

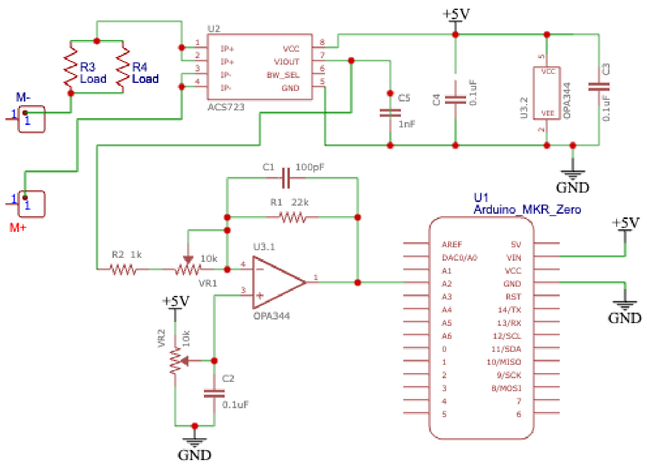
- **Attacker capabilities**
  - Has brief **physical access** to the target machine (e.g., cleaner, maintenance staff, insider).
  - Can insert a small USB/HDMI device into a **less visible** port.
  - No credentials, no OS-level privileges, no code on victim.
- **Device properties**
  - Mimics benign peripheral (display / low-power USB gadget).
  - Does not interfere with data channels or trigger warnings.
- **Attacker goals**
  - Infer running **CNN family / variant** on GPU.
  - Identify **rendered video clip** from a candidate set.
  - Potentially track **GPU-heavy malware** or crypto-mining.

Paper Contributions

1. **New side-channel:** Power leakage via **USB/HDMI ports** used to infer GPU workloads.
2. **Prototype device:** ACS723 + Arduino-based measurement board, sampling at up to **40 kHz effective rate**.
3. **Evaluation:**
   - CNN **architecture/family** inference.
   - **Video identification** from GPU-rendered clips.
4. **Discussion:** Limitations (proximity, background noise) and **defensive uses** of the same side-channel.

Hardware Setup & Circuit

- Core sensor: SparkFun ACS723 Hall-effect current sensor; MCU: Arduino MKR Zero (ARM Cortex-M0+), ADC configured for ≈ 80 kHz sampling; Load: two parallel 20 Ω film resistors to avoid tripping port protection and distribute heat.
- Ports monitored:
  - **USB 2.0** (5 V, up to 500 mA),
  - **HDMI pin 18** (5 V, $\leq$ 500 mA, powering HPD circuit).

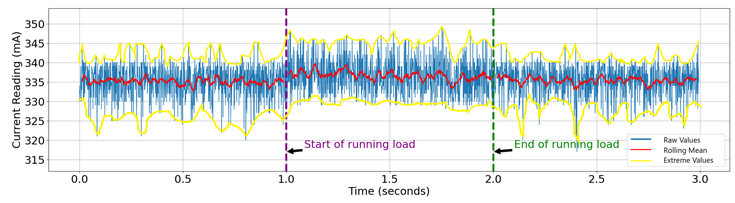


Schematic of the attack device capturing port current.

Preliminary Experiments

Goal: verify that GPU workloads induce measurable current fluctuations at USB/HDMI.

- **Glmark2 scene rendering:**
  - GPU-utilizing OpenGL scenes.
  - Both USB & HDMI traces show small but consistent **"humps"** aligned with rendering bursts.
- **Matrix multiplication on GPU:**
  - Multiply 1920×1080 matrices (matching 1080p resolution) using **CuPy**.
  - Tasks: $A \times B$ vs. $C \times D$, repeated 25 times.
  - Extracted features (e.g., mean, MFCC-2) form **distinct clusters** for the two multiplications on both ports.



Example raw traces and feature space separation.

CNN Inference Experiments

Objective: infer which CNN architecture (or family) is running on the GPU based solely on port power traces.

Models (families and variants):

- ConvNeXt, VGG, ResNet, Inception, MobileNet, DenseNet, NASNet, RegNet, EfficientNet, Xception.

Two configurations:

- **Config 1 (fully supervised):** Attacker has power data for each architecture variant.
- **Config 2 (family-level):** Attacker only sees one variant per family in training, must classify unseen variants into the correct **family**.

Image datasets:

- **Dataset 1:** 50 cats + 50 dogs (hard to classify pair).
- **Dataset 2:** 100 random ImageNet images.
- 80:20 train:test split, no overlap between sets.

Feature Extraction & ML Pipeline

From each time-series trace:

- **Statistical features** Sum, mean, variance, standard deviation, MAD, skewness, kurtosis, SEM, RMS, etc.
- **Spectral / audio-style features** (via Librosa):
  - Spectral centroid, bandwidth, flatness, roll-off,
  - MFCCs, tempo, chroma features, tempograms.

Feature selection:

- Scikit-learn `SelectKBest` to keep top 30 features.
- Highly informative: **sum**, spectral **roll-off**, **chroma** bins, **flatness**, **RMS**.
- Some CNN architectures form clearly distinct clusters in (sum, chroma-9) feature space.

Classifiers:

- k-Nearest Neighbors (KNN)
- Random Forest
- SVM (multiclass)
- XGBoost
- LightGBM

Best performer (most cases): KNN + `Normalizer` scaler, using Euclidean distance, $k = 5$, uniform weights.

Key Results – CNN Inference

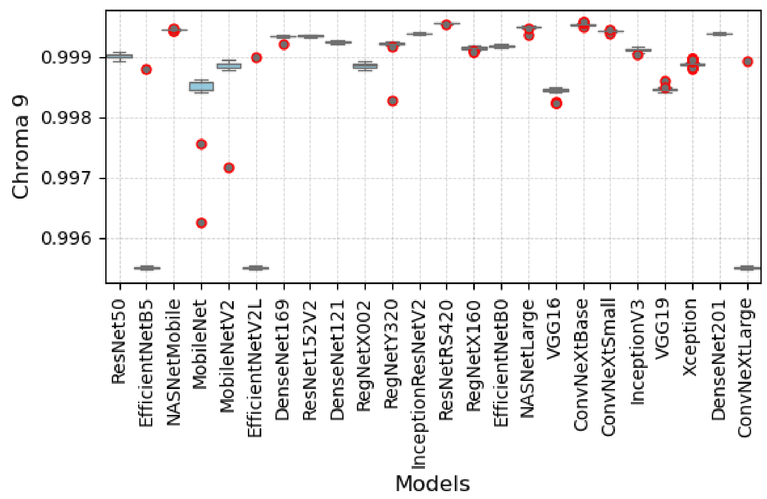
Base CNN identification (Configuration 1):

- **HDMI, Dataset 2:**
  - Accuracy: **94.25%**, F1: 0.94.
- **USB, unknown-class setting:**
  - Accuracy: up to **98.85%**, F1: 0.99.

Family-level inference (Configuration 2):

- **HDMI, Dataset 2:**
  - Accuracy: **76.45%** (regular), **74.39%** (with unknown class).
- **USB, Dataset 2:**
  - Accuracy: **72.10%** (regular), **71.39%** (with unknown class).

Misclassifications mainly occur between **architecturally similar** networks (e.g., VGG vs. MobileNet in some HDMI cases).



Feature importance: Chroma 9.

Video Rendering Inference

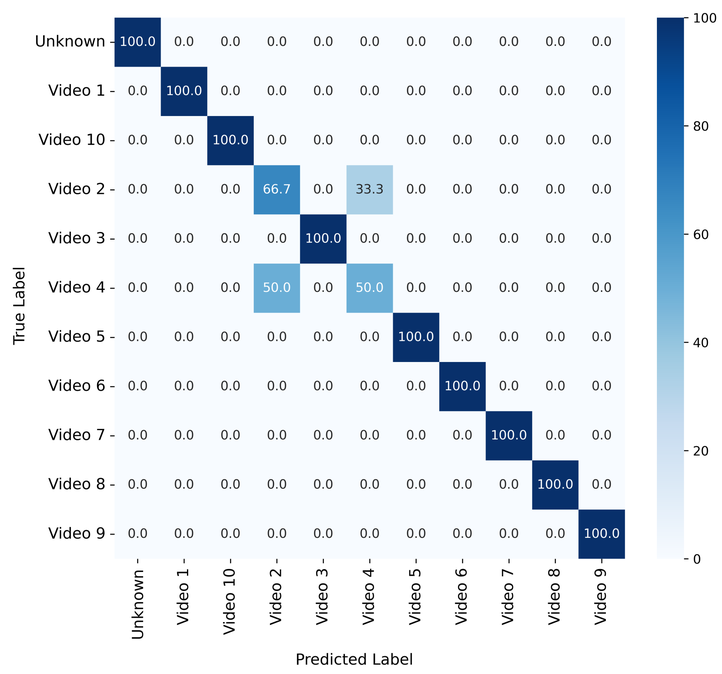
Goal: identify which of 10 candidate videos is being rendered.

- 10 different **H.264 MP4** videos (30–60 s).
- Each clip: play for 10s, then capture 10s of power data, repeat 25 times per video.
- Rendering controlled via **OpenCV** on GPU.

Features: similar pipeline as CNN experiments, with Chroma-9 and Flatness-0 particularly informative.

Results:

- **USB:** 92.86% accuracy, F1 = 0.92.
- **HDMI:** 100% accuracy (regular); 96.43% with unknown-class threshold.
- Most errors: confusion between visually similar clips.



Example confusion matrix for USB-based video inference.

Contact: `sarefin1@udayton.edu`